Object Oriented Programming

Polymorphism

Dr Raju Nadimpalli

Introduction to Polymorphism

- Polymorphism is one of the important feature of Object-Oriented Programming (OOP).
- Polymorphism means that you can have multiple classes where each class implements the same variables or methods in different ways.
- It refers to the use of a single type entity (method, operator or object) to represent different types in different scenarios.
    - A real-world example of polymorphism is suppose when if you are in classroom that time you behave like a student, when you are in market at that time you behave like a customer, when you are at your home at that time you behave like a son or daughter, such that same person is presented as having different behaviors.

Polymorphism in addition operator

- The + operator is used extensively in Python programs, have many usages.
- For integer data types, + operator is used to perform arithmetic addition operation.

```
num1, num2 = 10,20

print(num1+num2) # 30
```

- Similarly, for string data types, + operator is used to perform concatenation.

```
str1 = "Ten“

str2 = "Twenty“

print(str1+" "+str2) # Ten Twenty
```

- A single operator + has been used to carry out different operations for distinct data types.

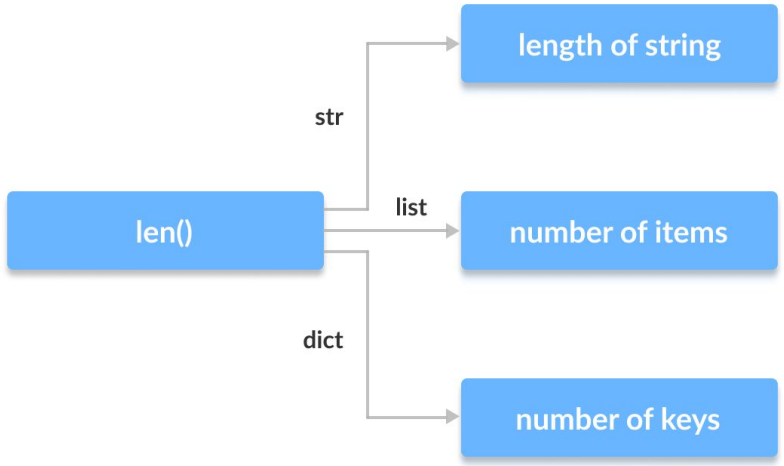
Function Polymorphism

- **There are some functions in Python which are compatible to run with multiple data types.**
- **One such function is the len() function. It can run with many data types in Python.**

```
print(len("Conduira Online"))   # 15

print(len(["Python", "Java", "C"]))  # 3

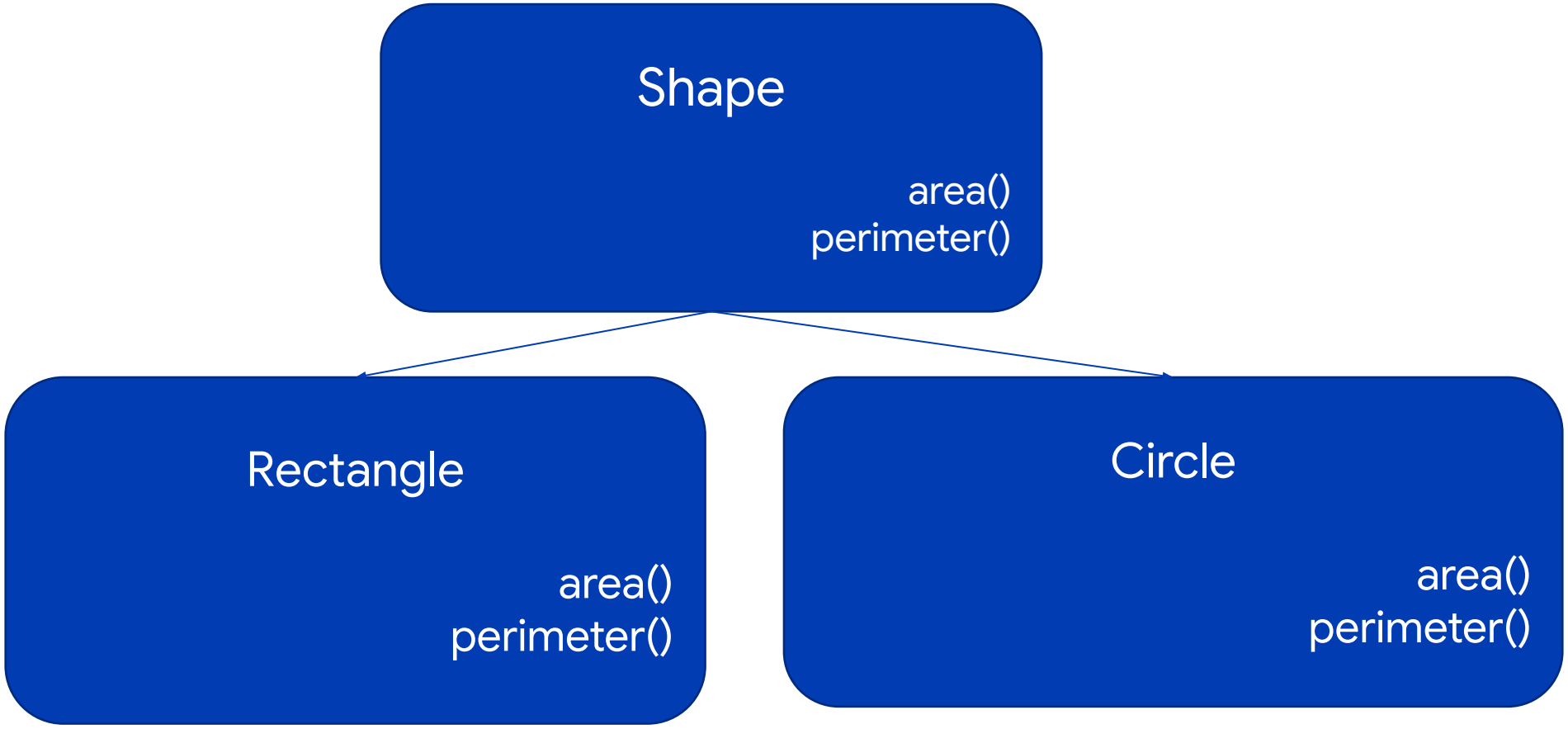
print(len({"Organization": "Conduira", "Address": "Hyderabad"}))
# 2
```



Note:

Data types such as string, list, tuple, set, and dictionary can work with the len() function. However, we can see that it returns specific information about specific data types.

Demonstrate Polymorphism



Demonstrate Polymorphism

```
import math
class Shape:
    def area(self):
        pass
    def perimeter(self):
        pass

class Rectangle(Shape):
    def __init__(self, width, height):
        self.width = width
        self.height = height
    def area(self):
        print(f"Area of Rectangle is {self.width * self.height}")
    def perimeter(self):
        print(f"Perimeter of Rectangle is {2 * (self.width + self.height)}")
class Circle(Shape):
    def __init__(self, radius):
        self.radius = radius
    def area(self):
        print(f"Area of Circle is {math.pi * self.radius ** 2}")

    def perimeter(self):
        print(f"Perimeter of Circle is {2 * math.pi * self.radius}")
```

```
rectangle_obj = Rectangle(10, 20)
rectangle_obj.area()
rectangle_obj.perimeter()
print()
circle_obj = Circle(10)
circle_obj.area()
circle_obj.perimeter()
```

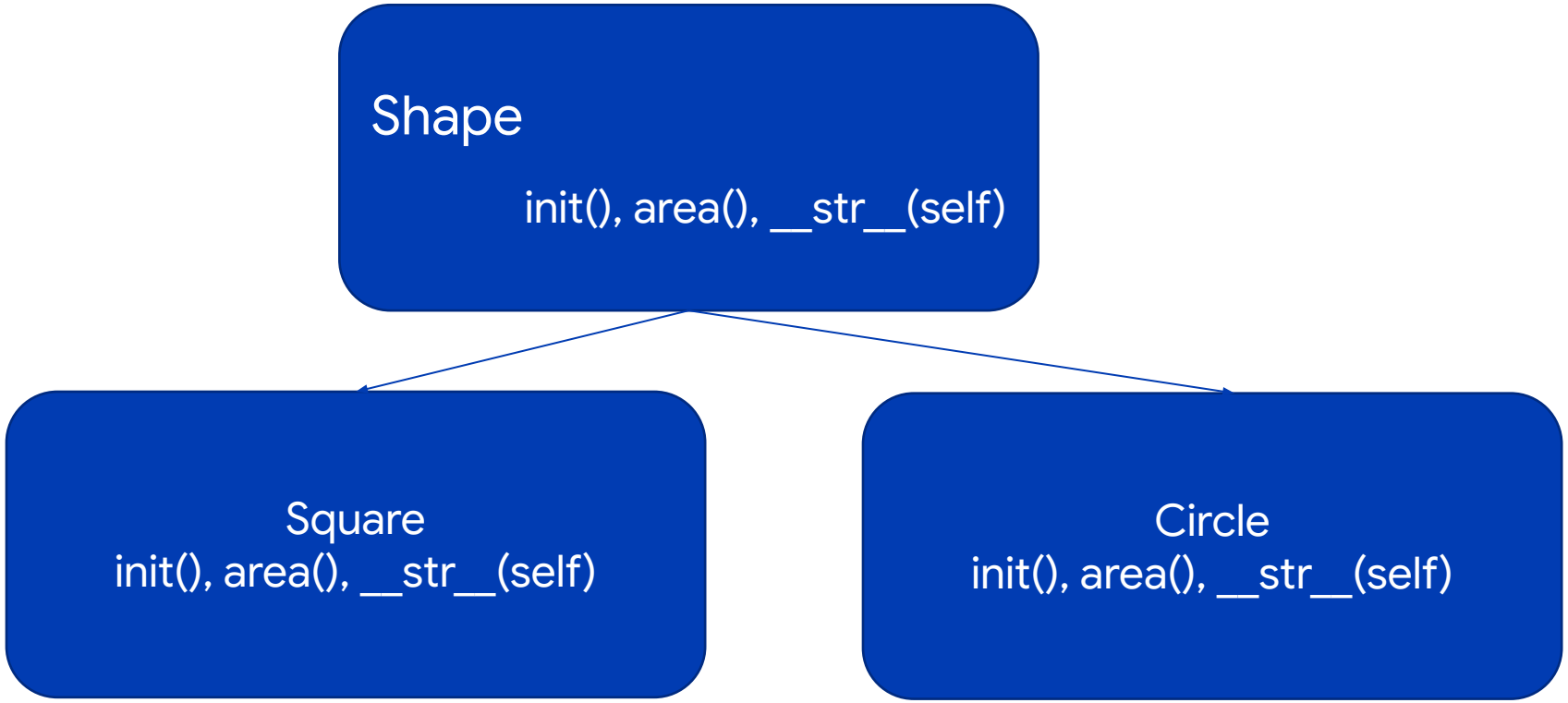
Explanation

- Shape is the base class while Rectangle and Circle – are the derived classes.
- All these classes have common methods area() and perimeter() added to them but their implementation is different as found in each class.
- Derived classes Rectangle and Circle have their own data attributes.
- Instance variables rectangle_obj and circle_obj are created for Rectangle and Circle classes respectively.
- The clearest way to express polymorphism is through the function shape_type() – , which takes any object and invokes the methods area() and perimeter() respectively.

Method Overriding

- Like in other programming languages, the child classes in Python also inherit methods and attributes from the parent class.
- We can redefine certain methods and attributes specifically to fit the child class, which is known as **Method Overriding**.
- Polymorphism allows us to access these overridden methods and attributes that have the same name as the parent class.

Method Overriding



Method Overriding

```
from math import pi

class Shape:
    def __init__(self, name):
        self.name = name

    def area(self):
        pass

    def __str__(self):
        return self.name

class Square(Shape):
    def __init__(self, length):
        super().__init__("Square")
        self.length = length

    def area(self):
        return self.length**2
```

Method Overriding

```
class Circle(Shape):
    def __init__(self, radius):
        super().__init__("Circle")
        self.radius = radius

    def area(self):
        return pi*self.radius**2


a = Square(4)
b = Circle(7)
print(a)
print(a.area())
print(b)
print(b.area())
```

```
Square
16
Circle
153.93804002589985
```

The methods __str__(), which have not been overridden in the child classes, are used from the parent class.